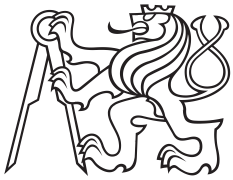
Bachelor Project



Czech Technical University in Prague

F3

Faculty of Electrical Engineering
Department of Radioelectronics

3D Mapping Based on LiDAR Signal Processing

Illia Novosad

Supervisor: Ing. Pavel Puričer, Ph.D.
Field of study: Electronics and Communications
May 2026

ZADÁNÍ BAKALÁŘSKÉ PRÁCE

I. OSOBNÍ A STUDIJNÍ ÚDAJE

Příjmení: **Novosad** Jméno: **Illia** Osobní číslo: **526428**

Fakulta/ústav: **Fakulta elektrotechnická**

Zadávající katedra/ústav: **Katedra radioelektroniky**

Studijní program: **Elektronika a komunikace**

II. ÚDAJE K BAKALÁŘSKÉ PRÁCI

Název bakalářské práce:

Zpracování signálu Lidaru pro 3D mapování

Název bakalářské práce anglicky:

3D Mapping Based on Lidar Signal Processing

Jméno a pracoviště vedoucí(ho) bakalářské práce:

Ing. Pavel Puričer, Ph.D. katedra radioelektroniky FEL

Jméno a pracoviště druhé(ho) vedoucí(ho) nebo konzultanta(ky) bakalářské práce:

Datum zadání bakalářské práce: **02.02.2026** Termín odevzdání bakalářské práce: _____

Platnost zadání bakalářské práce: **19.09.2027**

podpis vedoucí(ho) ústavu/katedry

podpis proděkana(ky) z pověření děkana(ky)

III. PŘEVZETÍ ZADÁNÍ

Student bere na vědomí, že je povinen vypracovat bakalářskou práci samostatně, bez cizí pomoci, s výjimkou poskytnutých konzultací. Seznam použité literatury, jiných pramenů a jmen konzultantů je třeba uvést v bakalářské práci.

Datum převzetí zadání

Novosad Illia

Podpis studenta

ZADÁNÍ BAKALÁŘSKÉ PRÁCE

I. OSOBNÍ A STUDIJNÍ ÚDAJE

Příjmení: **Novosad** Jméno: **Illia** Osobní číslo: **526428**

Fakulta/ústav: **Fakulta elektrotechnická**

Zadávající katedra/ústav: **Katedra radioelektroniky**

Studijní program: **Elektronika a komunikace**

II. ÚDAJE K BAKALÁŘSKÉ PRÁCI

Název bakalářské práce:

Zpracování signálu Lidaru pro 3D mapování

Název bakalářské práce anglicky:

3D Mapping Based on Lidar Signal Processing

Pokyny pro vypracování:

There are used various methods based on the evaluation of the ranging signal to map the topology of the environment. Study these methods with a focus on the use of lidar signals. Based on the study results, design a method and algorithms for using a directional beam lidar module or an omnidirectional lidar module for scanning a 3D space. Assume that the lidar will be mounted on a platform that will either be linearly lifted, angularly tilted and/or rotated. The goal is to design the processing of the output signal of the lidar module so that a 3D image of the scanned space can be displayed and propose the construction solution of the lidar installation.

Seznam doporučené literatury:

[1]Pinliang Dong, Qi Chen: LiDAR Remote Sensing and Applications. CRC Press, Boca Raton 2018, ISBN 978-1-138-74724-1
[2]Theodor Sjöstedt: Lidar Signal Processing Techniques: Clutter Suppression, Clustering and Tracking. [online]. Master's Degree Project, KTH SIGNAL PROCESSING, Stockholm 2011. Dostupné z https://www.diva-portal.org/smash/get/diva2:470911/FULLTEXT01.pdf [cit. 2025-06-01].

FAKULTA ELEKTROTECHNICKÁ
FACULTY OF ELECTRICAL ENGINEERING
Technická 2
166 27 Praha 6

DECLARATION

I, the undersigned

Student's surname, given name(s):	Novosad Illia
Personal number:	526428
Programme name:	Electronics and Communications

hereby declare that the Bachelor's Thesis titled

3D Mapping Based on Lidar Signal Processing

is my own independent work and that I have declared all sources of information used in accordance with the Methodological Guideline for adhering to ethical principles when elaborating an academic final thesis and the Framework Rules for the use of artificial intelligence at CTU for study and pedagogical purposes in Bachelor and continuing Masters studies.

I declare that I have used artificial intelligence tools during the preparation and writing of the final thesis.
I've verified the generated content. I confirm that I am aware that I am fully responsible for the content of the final thesis.

In Prague on 17.05.2026

Illia Novosad
.....
student's signature

Acknowledgements

I am deeply grateful to my supervisor, Ing. Pavel Puričer, Ph.D., for his time and guidance during the work on this thesis.
I would also like to thank my colleague Viktor Trachta for the valuable suggestions regarding the mechanical design and for providing last-minute help with the components for the platform.
My greatest thanks go to my family for their support and encouragement throughout my studies and for giving me the opportunity to pursue my passion for electronics at CTU.

Declaration

I hereby declare that artificial intelligence tools were used during the writing of this thesis.
ChatGPT was used as an aid in generating TikZ figures (at the time of writing the latest version was 5.5).
Copilot was used as an integrated autocompletion tool for small chunks of code inside Visual Studio Code, and ChatGPT was used to identify issues related to edge cases that the initial code did not cover and suggest improvements.
Sentence structure was checked with ChatGPT.
Grammar was checked with Grammarly.
All generated content was verified, and I take full responsibility for the contents of this project.
All information taken from the work of other authors has been properly cited.

In Prague, 22. May 2026

Abstract

A wide range of Light Detection and Ranging (LiDAR) sensors for spatial scanning is currently available on the market; however, most of them remain too expensive for amateur use. The aim of this thesis is to provide an overview of existing DIY solutions, evaluate their limitations, and propose a platform design that addresses the identified weaknesses. A working prototype of the proposed design is then presented along with a description of the design process. The transmission of data from the LiDAR sensor to a PC is then described, and the code used for controlling the platform and acquiring the data is presented. The resulting point cloud is then presented and discussed, including the code corrections introduced to compensate for the design flaws and the possible internal misalignments of the LiDAR sensor. Finally, several possible design improvements are proposed for further development of this project.

Keywords: LiDAR, 3D mapping, point cloud, spatial scanning, Open3D, Raspberry Pi Pico, planetary gearbox

Supervisor: Ing. Pavel Puričer, Ph.D.

Abstrakt

Dnes je na trhu široká škála senzorů pro prostorové skenování; většina z nich však zůstává pro amatérské využití příliš drahá. Cílem této práce je podat přehled existujících DIY řešení, zhodnotit jejich omezení a navrhnout konstrukci platformy, která řeší identifikované nedostatky. Funkční prototyp je následně představen spolu s popisem procesu návrhu. Dále je popsán přenos dat ze senzoru LiDAR do počítače a je prezentován kód použitý pro řízení platformy a sběr dat. Výsledný point cloud je následně prezentován a diskutován, včetně drobných úprav kódu zavedených za účelem kompenzace konstrukčních chyb návrhu a možných vnitřních nepřesností senzoru LiDAR. Na závěr jsou navržena možná vylepšení konstrukce pro další vývoj tohoto projektu.

Klíčová slova: LiDAR, 3D mapování, mračno bodů, prostorové skenování, Open3D, Raspberry Pi Pico, planetová převodovka

Překlad názvu: Zpracování signálu Lidaru pro 3D mapování

Contents

Acronyms 3

1 Introduction 5

2 Design of a Platform 7

2.1 Existing Amateur Solutions for Environmental Mapping 7

2.1.1 Using a Single-Point LiDAR module 7

2.1.2 Using a Single-Point LiDAR and a Mirror 8

2.1.3 Using a 2D LiDAR Mounted on a Tilting Platform 9

2.2 Proposed Design of a 360° LiDAR Platform 10

2.2.1 Controlling the Platform 12

2.2.2 Choosing the Tilt Angle 13

2.2.3 Justification for Gearbox Usage 15

2.3 Gearbox Design 17

2.3.1 Choosing reduction ratio 17

2.3.2 Gearbox Design Selection . . . 17

2.3.3 Choosing Teeth Count 20

2.4 Final Platform Design 22

3 Electronics and Software Implementation 25

3.1 Electronic Design 25

3.2 Software Implementation 26

3.2.1 Pico program 26

3.2.2 PC program 27

3.2.3 Point cloud reconstruction . . 28

4 Experimental Results 31

4.1 Scanning procedure 31

4.2 Point Cloud Processing 31

4.2.1 Outlier Removal 31

4.2.2 Misalignment Corrections . . . 33

4.2.3 Comparison with Photograph 35

5 Conclusion 37

A Bibliography 39

B Attached files 41

B.1 code.zip 41

B.2 3d_models.zip 42

B.3 Project Repository 43

Figures

1.1 Photo of STL-19P LiDAR [1] 5

2.1 Implementation in project [5] using a single-point LiDAR 8

2.2 Implementation in project [6] using a single-point LiDAR and a mirror 9

2.3 Implementation in project [7] using a tilting platform 9

2.4 Simulation of an obtained point cloud using a tilting platform . . 10

2.5 Proposed platform design. θ is the tilt angle 11

2.6 Simulation of an obtained point cloud using a rotating tilted platform with two different tilt angles θ 12

2.7 NEMA 17 17HS8401 photo[2] . . . 13

2.8 Visualisation of scan spacing dependence on the tilt angle . . . 14

2.9 Scan spacing depending on the tilt angle θ 14

2.10 Side profile of a point cloud obtained using a rotating tilted platform with tilt angle $\theta = 23.4^\circ$ 15

2.11 Point cloud resolution depending on the tilt angle θ and gear ratio 16

2.12 Belt drive featured in [10] 17

2.13 Photo of the components of cycloidal gearbox. Illustration taken from [12] 18

2.14 Section view of a two-stage planetary gearbox. Illustration taken from [15] 19

2.15 Visualisation of the geometric constraint defined by eq. (2.3)[6]. Gears are represented as circles for clarity 21

2.16 Complete assembly of a platform in Onshape. Project link 22

2.17 Exploded view of the platform 23

2.18 Photo of the assembled platform 24

3.1 Block diagram of the electronic system. Dashed arrows represent power connections; solid arrows represent signal or control connections 25

3.2 Visualisation of the tilt transformation 29

3.3 Visualisation of the azimuth transformation 30

4.1 Raw point cloud 32

4.2 Visualisation of the outlier removal process 32

4.3 Correction of the ceiling misalignment shown in side view 34

4.4 Correction of the wall misalignment shown in side view 35

4.5 Comparison between a photo and the point cloud of a scanned room 36

Tables

1.1 Main parameters of STL-19P LiDAR[1] 6

2.1 Crucial parameters of the chosen NEMA 17 17HS8401 stepper motor[1] 13

2.2 Best combinations for step angle/gear ratio/tilt angle 16

5.1 Bill of materials 38

Acronyms

LiDAR Light Detection and Ranging

DSP Digital Signal Processing

SLAM Simultaneous Localisation and Mapping

UART Universal Asynchronous Receiver-Transmitter

Chapter 1

Introduction

Nowadays, LiDAR sensors are widely used in various applications, including autonomous vehicles, robotics, and environmental mapping [1; 2, Ch. 4-6]. While the former two applications often require high-end and expensive LiDAR systems, environmental mapping can be effectively performed using more affordable sensors. However, even the cheapest commercial 3D LiDAR sensor, which was found during the research for this thesis, costs 249 € [3].

This bachelor's thesis addresses two main objectives. The first objective is the development of a cost-effective platform for a 360 ° STL-19P LiDAR sensor that enables 3D spatial scanning using a 2D LiDAR sensor.

Figure 1.1: Photo of STL-19P LiDAR [1]

The second objective is the utilisation of hardware tools, such as the Raspberry Pi Pico 2 W microcontroller[2] for data acquisition, and software

[1] Photo and parameters were taken from `https://www.waveshare.com/wiki/D500_LiDAR_Kit`

[2] `https://www.raspberrypi.com/products/raspberry-pi-pico-2/`

Table 1.1: Main parameters of STL-19P LiDAR[1]

Typical measuring range [m]	0,03–12
Angular resolution [°]	0,72
Dimensions (W x L x H) [mm, mm, mm]	$38,59 \times 38,59 \times 33,50$
Communication interface	UART @ 230400 bps
Rotational frequency [Hz]	10
Sampling frequency [Hz]	5000

tools, such as the Python library Open3D [4] for the 3D scene reconstruction.

Chapter 2

Design of a Platform

This chapter will firstly go over the existing DIY solutions for 360° LiDAR platforms discovered during the research for this project, analysing their pros and cons. Subsequently, the proposed design is presented, along with the chosen design decisions and their justifications.

2.1 Existing Amateur Solutions for Environmental Mapping

2.1.1 Using a Single-Point LiDAR module

One of the most straightforward approaches to achieve 360° environmental mapping is to utilise a single-point LiDAR module mounted on a platform which can rotate and tilt. The implementation could be found in [5].

Pros	Cons
+ Single-point LiDARs are the most cost-effective among all existing types	− Slow data acquisition relative to methods leveraging 2D LiDAR's speed
+ Simple design	− Movement of lidar may introduce inaccuracies in distance measurements, which need to be compensated for in code if we want to assume that all points are measured from a single position in space

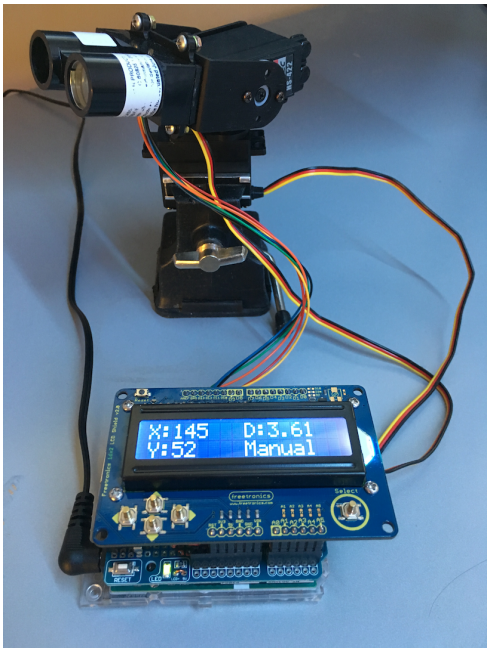


Figure 2.1: Implementation in project [5] using a single-point LiDAR

2.1.2 Using a Single-Point LiDAR and a Mirror

The following approach was used in [6]. A single-point LiDAR is placed below a rotating mirror, which reflects the laser beam horizontally. As the mirror rotates, the laser beam scans the environment in 360-degree sweep. Moreover, by tilting the mirror at an angle, vertical scanning can also be achieved, allowing for the acquisition of the whole scene.

Pros

+ Cost-effectiveness as in previous implementation
+ Simple design
+ Much easier to correct the distance measurement since the LiDAR is stationary and the distance from it to the mirror is known.

Cons

– Slow data acquisition relative to methods leveraging 2D LiDAR's speed
– Mirror quality may affect the accuracy of distance measurements
– Mechanical structure holding the mirror may introduce vibrations, thus affecting measurement accuracy

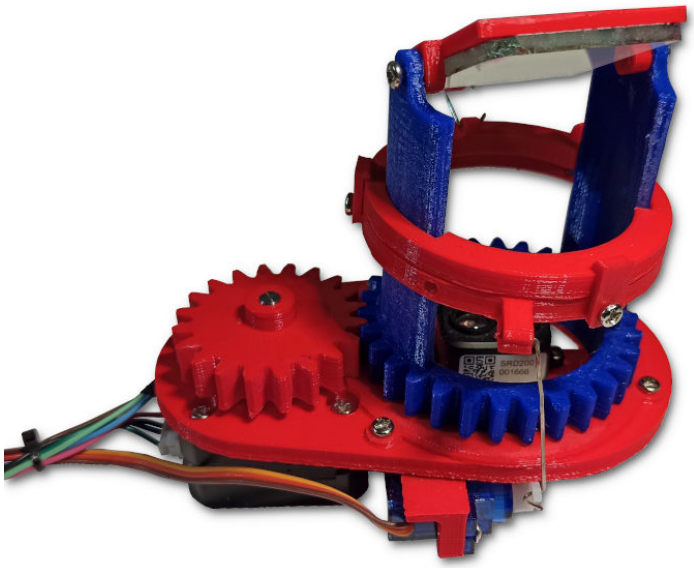


Figure 2.2: Implementation in project [6] using a single-point LiDAR and a mirror

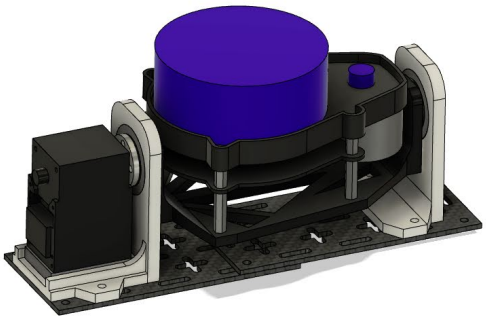


Figure 2.3: Implementation in project [7] using a tilting platform

2.1.3 Using a 2D LiDAR Mounted on a Tilting Platform

This approach utilises a 2D LiDAR sensor mounted on a platform that can tilt the sensor side to side, as demonstrated in [7]. The 2D LiDAR scans the environment in a horizontal plane, while the tilting mechanism allows for vertical scanning.

Pros

+ Faster data acquisition compared to solutions with single-point LiDAR measurements
+ Higher resolution given by the use of a 2D LiDAR sensor
+ This design could be potentially used for Simultaneous Localisation and Mapping (SLAM) applications, given the higher speed

Cons

– More expensive than single-point LiDAR solutions
– Point distribution is non-uniform due to the tilting mechanism, which produces denser point clouds on the axis of tilting (see fig. 2.4)
– Mechanical design is more prone to vibrations produced by the LiDAR itself, affecting measurement accuracy

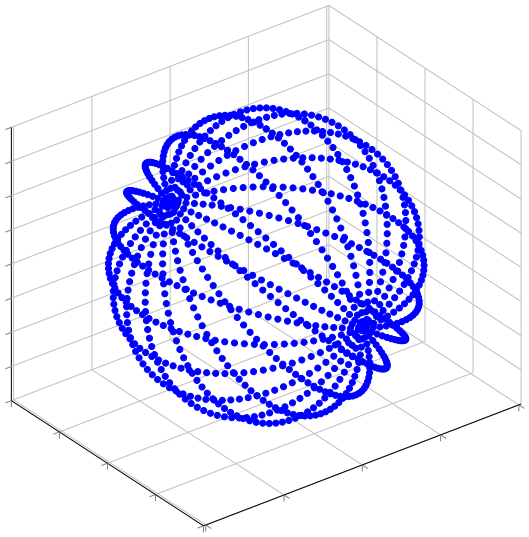


Figure 2.4: Simulation of an obtained point cloud using a tilting platform

2.2 Proposed Design of a 360° LiDAR Platform

Based on the analysis of existing DIY solutions for 360° LiDAR platforms, the proposed design for this miniproject involves using a 2D LiDAR sensor mounted on a tilted platform that is able to rotate around its axis.

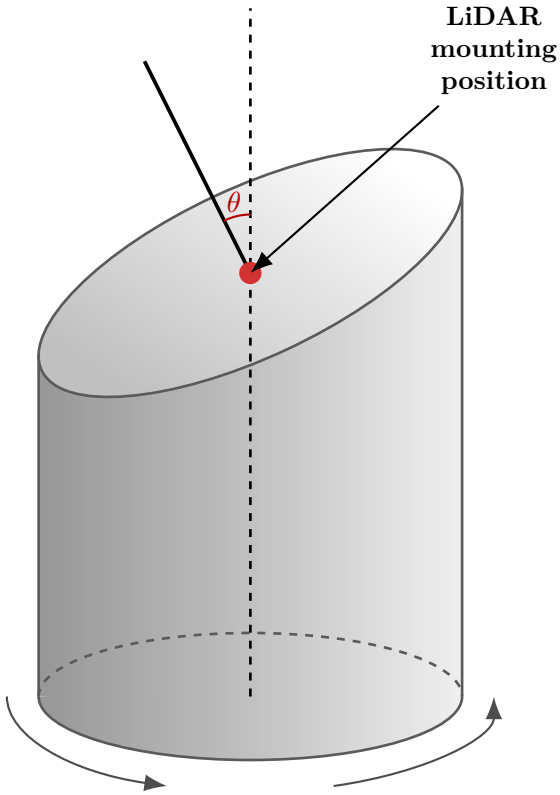


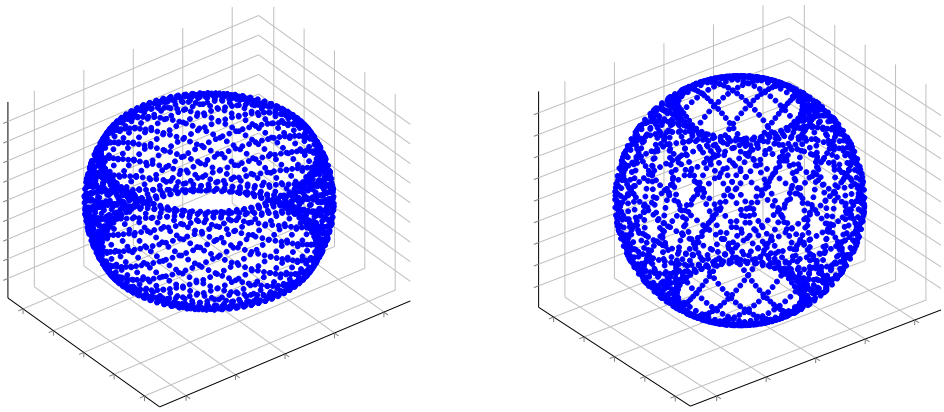
Figure 2.5: Proposed platform design. θ is the tilt angle

This design leverages the advantages of 2D LiDAR sensors, such as fast data acquisition and high resolution, while also eliminating some of the drawbacks of previous designs, namely:

+ The platform will be pretty rigid, which will minimise the influence of vibrations
+ Mechanical design is very simple, making it easy to build and troubleshoot
+ The resulting point cloud will be more uniformly distributed compared to the one obtained using the tilting platform design presented in section 2.1.3 unless the tilt angle is extremely steep (see fig. 2.6)

Such a design does have its drawbacks, however. The first one is immediately apparent from fig. 2.6: there are blind spots in the point cloud where no points are measured. While these blind spots can be minimised by choosing a large tilt angle, this approach reintroduces the problem of non-uniform point cloud density.

Besides that, compared to the tilting platform design presented in section 2.1.3, the proposed design would need to rotate across twice as large a



(a) : $\theta = 30°$ **(b) :** $\theta = 60°$

Figure 2.6: Simulation of an obtained point cloud using a rotating tilted platform with two different tilt angles θ

range to obtain full coverage of the environment, since the tilting platform scanned only within $[-90°, +90°]$, whereas the current design must complete a full 360-degree rotation. Even though this surely leads to longer scanning times, the obtained point cloud will be more dense.

2.2.1 Controlling the Platform

Since we want the platform to be precisely controlled, we need to choose an appropriate motor. Stepper motors are commonly used in applications requiring precise positioning, as they can be controlled to move in small, discrete steps. For this design, a NEMA 17 17HS8401 stepper motor is used to rotate the LiDAR platform.

Figure 2.7: NEMA 17 17HS8401 photo[1]

Table 2.1: Crucial parameters of the chosen NEMA 17 17HS8401 stepper motor[1]

Motor Weight [g]	360
Holding Torque [N m]	0,5
Step Angle [°]	1,8
# of Steps per Revolution [-]	200
Rated Current [A]	1,68

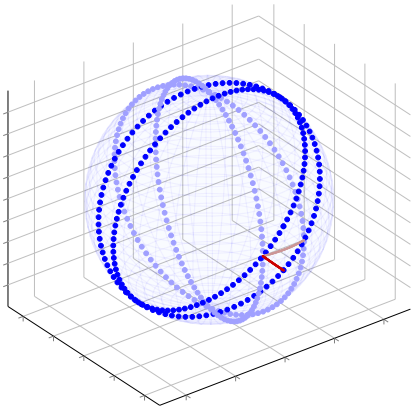
For heavy load applications, it is important to consider the potential error sources of stepper motors, such as the stepper motor losing steps due to a heavy load or by driving the motor faster than its maximum speed [8, p. 13]. However, in our case, the load will be relatively light for this motor, the speed will be kept low, and a gearbox mechanism will be utilised which will both reduce the step angle and increase torque; thus, a feedback mechanism for correcting lost steps is not required.

2.2.2 Choosing the Tilt Angle

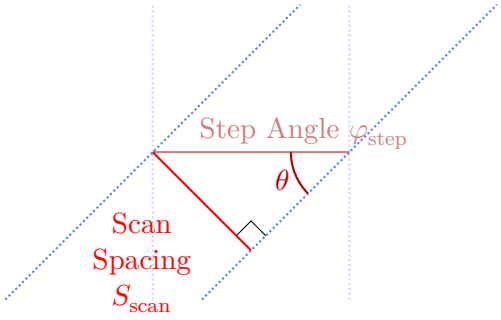
The tilt angle of the LiDAR platform is a crucial parameter that affects the size of the aforementioned blind spots in the point cloud (see fig. 2.6). However, increasing the tilt angle also increases the spacing between the scans when the platform is rotated in discrete steps (see fig. 2.8). It is obvious from the fig. 2.8b, that

$$S_{\text{scan}} = \varphi_{\text{step}} \cdot \sin(\theta), \tag{2.1}$$

[1]Photo and parameters were taken from `https://www.laskakit.cz/en/krokovy-motor-nema-17-17hs8401-0-5nm/`



(a) : 3D simulation



(b) : Simplified image showing the dependence of scan spacing S_{scan} on tilt angle θ

Figure 2.8: Visualisation of scan spacing dependence on the tilt angle

where the step angle φ_{step} is constant and is equal to 1.8° for the chosen motor (see table 2.1). The following graph shows how the spacing changes with the tilt angle, providing our LiDAR's resolution as a reference value, which we would like to achieve by choosing a correct tilt angle:

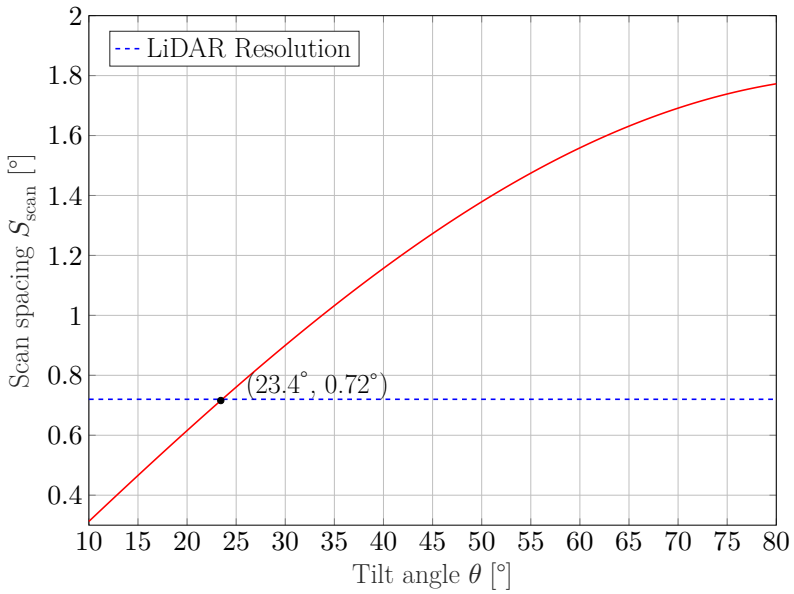


Figure 2.9: Scan spacing depending on the tilt angle θ

As could be seen from fig. 2.9, the spacing will be the same as the angular resolution of the LiDAR itself at the tilt angle of approximately 23 4°.

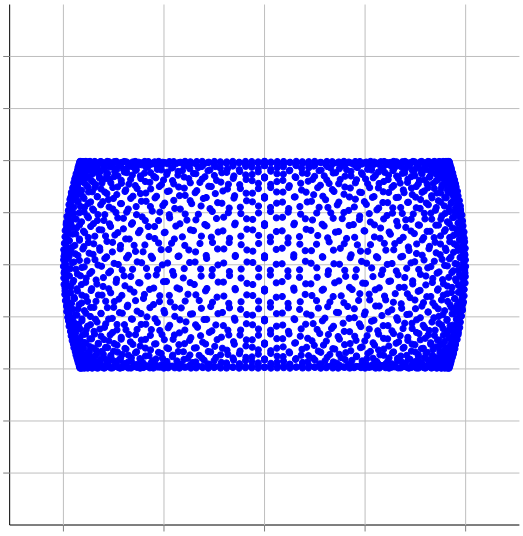


Figure 2.10: Side profile of a point cloud obtained using a rotating tilted platform with tilt angle $\theta = 23.4^{\circ}$

By examining the side profile of the point cloud obtained using this tilt angle, extremely large blind spots can be observed, making this tilt angle impractical for our application. Fortunately, there is a way to achieve a more complete coverage of the environment without sacrificing point cloud density.

2.2.3 Justification for Gearbox Usage

To eliminate the need for a compromise between point cloud coverage and density when using a stepper motor to rotate the platform, we can incorporate a gearbox mechanism into the design. Using gears with a specific gear ratio allows us to choose a tilt angle that minimises blind spots while still producing a dense point cloud. The resulting spacing between scans can be expressed as

$$S_{\text{scan}} = \frac{\varphi_{\text{step}} \cdot \sin(\theta)}{R}, \tag{2.2}$$

where R is the output step reduction ratio of the gearbox, i.e. the the ratio between the step angle of the input and the corresponding step angle of the output. A following graph is similar to fig. 2.9, but the third dimension represents different gear ratios:

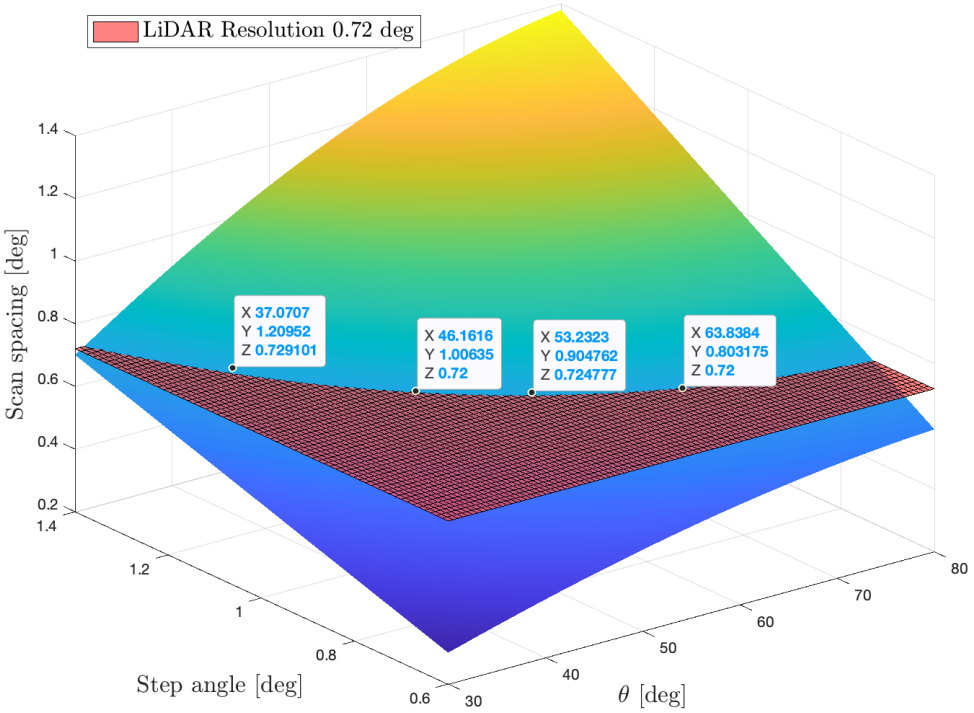


Figure 2.11: Point cloud resolution depending on the tilt angle θ and gear ratio

As can be seen from fig. 2.11, using a gearbox allows us to choose from a wide range of tilt angles without compromising on the spacing between scans.

Table 2.2: Best combinations for step angle/gear ratio/tilt angle

Step Angle [°]	Gear Ratio [-]	Tilt Angle [°]
1,2	3:2	37
1,0	9:5	46
0,9	2:1	53
0,8	9:4	64

Of course, by using the gearbox, we increase the number of steps required for a complete revolution, thereby increasing the scanning time. Nevertheless, scanning time is not that long in the first place to be a deciding factor when designing a platform, making such design choice a reasonable solution for increasing point cloud coverage while preserving the density.

2.3 Gearbox Design

2.3.1 Choosing reduction ratio

The step angle of the stepper motor is equal to 1.8° (see table 2.1). An angle of 0.1° was selected as the base output step angle of the platform, since it lets all angles featured in table 2.2 be represented as integer multiples of this value, meaning a larger step will be artificially achieved by performing multiple small steps. This requires an overall reduction ratio of 18:1. DRV8825 stepper motor driver[2] used in this project supports microstepping up to 32:1. A microstepping setting of 2:1 was selected, since higher microstepping ratios can significantly reduce positioning accuracy, especially for the DRV8825 driver [9, sec. Texas Instruments DRV8825]. That remaining 9:1 reduction ratio is therefore achieved mechanically using a gearbox.

2.3.2 Gearbox Design Selection

Belt Drive

A belt drive is a mechanical transmission system in which rotational motion is transferred between pulleys using a flexible belt. The ratio of pulley diameters determines the reduction ratio. In case of toothed belts, gears could be used instead of pulleys to minimise slipping. An example of a 3D-printed belt drive mechanism can be found in [10, sec. Belt Drive]

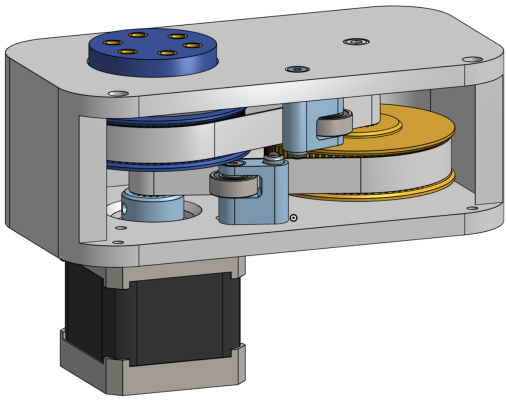


Figure 2.12: Belt drive featured in [10]

[2] https://www.ti.com/lit/ds/symlink/drv8825.pdf

Pros	Cons
+ Using belt ensures smooth and quiet operation that is hard to achieve with direct teeth-to-teeth contact in interlocking gears	– The asymmetric mechanical arrangement, although compact overall, could obstruct the field of view of the LiDAR sensor unless the scanning platform positions it excessively high above the drive mechanism. The symmetric designs presented next do not require such an elevated platform.

Cycloidal Gearbox

A cycloidal gearbox is widely used in robotics and machining due to its low backlash[3] and its ability to provide exceptionally high reduction ratios and torque in a compact form factor [11]. A 3D-printed implementation and further discussion of its properties can be found in [12].

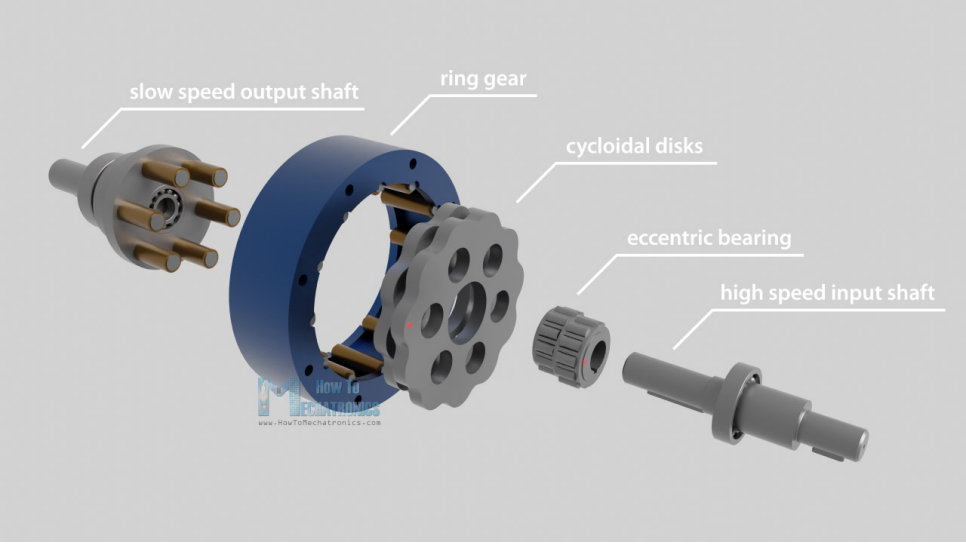


Figure 2.13: Photo of the components of cycloidal gearbox. Illustration taken from [12]

[3](dict.)Backlash – the play between adjacent movable parts (as in a series of gears)

Pros	Cons
+ Symmetric design (a requirement for our project)	– Correct design of this system requires working with complex parametric equations inside CAD software to correctly design the movement of disks in a cycloidal manner (hence the name). Although CAD tutorials and design tools exist [13], the design process remains cumbersome.
+ High reduction ratio in a compact space	– High component count due to the large number of rollers required for high reduction ratios. In our case, a cycloidal gearbox with a 9:1 ratio would require 10 ball bearings for each of the two cycloidal disks, making the design unnecessarily complex for an amateur project.

Planetary gearbox

A planetary gearbox is a compact symmetric transmission system consisting of a sun gear, planet gears, and an outer ring gear. They are widely used in automotive transmissions because multiple gear ratios can be achieved within a compact arrangement by selectively locking or driving different transmission components [14, p. 158-159]. 3D-printed implementation can be seen in [15].

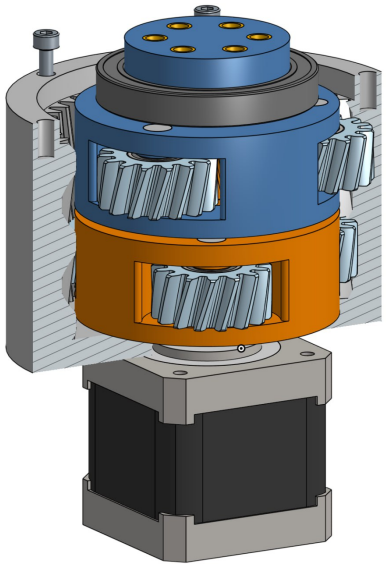


Figure 2.14: Section view of a two-stage planetary gearbox. Illustration taken from [15]

Pros	Cons
+ Symmetric, simple and compact design	– The backlash and noise of the gearbox are strongly influenced by the printing accuracy[5].
+ Smooth operation can be achieved using helical gears[4].	

All things considered, a planetary gearbox is the best solution for our application, featuring both symmetric arrangement of components and simplicity while still providing favourable mechanical characteristics.

2.3.3 Choosing Teeth Count

Unlike with a cycloidal gearbox, it is harder to achieve 9:1 reduction ratio only with one stage while still keeping the overall design compact. Thus, a two-stage design is needed.

Mathematically, planetary gearbox has to satisfy the following equations [16]:

$$Z_{\text{ring}} = Z_{\text{sun}} + 2 * Z_{\text{planet}}, \tag{2.3}$$

where $Z_{\text{ring}}, Z_{\text{sun}}$ and Z_{planet} denote the number of teeth of the respective gears. The number of teeth is linearly proportional to the pitch diameter of a gear since they are located at the circumference. Thus, this equation represents a purely geometry constraint required for the sun and ring gears to be able to fit inside the ring gear.

[4]The potential drawback in some designs could be the axial forces which arise due to helical nature of teeth. However, for our project this is not an issue since the design of gearbox is vertical.

[5]In this project, gears with larger teeth were used in order to reduce the influence of printing tolerances. Although the result shows almost no backlash, it remains quite noisy.

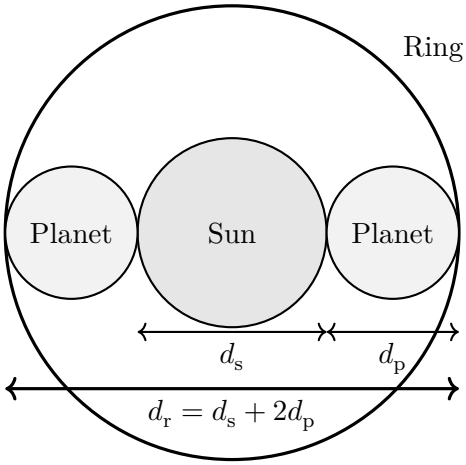


Figure 2.15: Visualisation of the geometric constraint defined by eq. (2.3)[6]. Gears are represented as circles for clarity

The next equation that all planet gears can be placed with equal angular spacing around the sun gear. Equal spacing ensures equal load distribution and helps balance the forces with which planet gears act upon the sun gear.

$$\frac{Z_{\text{ring}} + Z_{\text{sun}}}{N_{\text{planets}}} = k, k \in \mathbb{N}. \tag{2.4}$$

If this condition is not fulfilled, the planet gears cannot be equally spaced because proper tooth alignment between the planet gears and the sun/ring gears would not be maintained.

As was already mentioned in section 2.3.2, planetary gearboxes could output different gear ratios depending on which component is fixed and which component is used as the input. In our case, a ring gear will be part of the outer shell of the platform, thus it will be stationary. In our case, the following transmission ratio will be true [17, sec. Fixed ring gear]:

$$R = 1 + \frac{Z_{\text{ring}}}{Z_{\text{sun}}}. \tag{2.5}$$

Considering the constraints given in the eqs. (2.3) and (2.4), for a transmission ratio of 3:1 for one stage, the following teeth counts were chosen:

$$Z_{\text{ring}} = 64, Z_{\text{sun}} = 32, Z_{\text{planet}} = 16. \tag{2.6}$$

[6]Two planet gears were used for illustration purposes. Our design uses three planet gears, but the constraint condition remains the same.

2.4 Final Platform Design

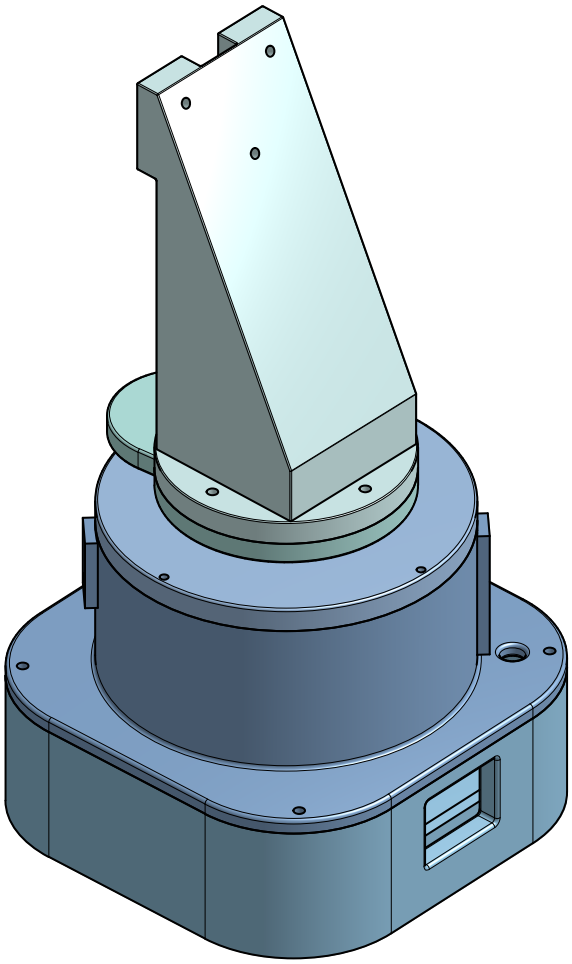


Figure 2.16: Complete assembly of a platform in Onshape. Project link

For the testing purposes, the platform with the tilt angle of 64° was designed. The exploded view shown in fig. 2.17 illustrates all the components of the platform and their arrangement.

The hole located on the underside of the platform mount is intended for the insertion of a magnet used for the platform homing procedure. Two rectangular extensions positioned on opposite sides of the ring gear are used for mounting the microcontroller and the Hall effect sensor. Two holes are present in the base of the ring gear. The central hole is meant for the insertion of the stepper motor shaft, while the hole located near one of the rectangular extensions is used for routing wires supplying power to the microcontroller and the LiDAR sensor, as well as wires used for controlling the stepper motor driver.

Item	Quantity	Description
1	1	Platform
2	1	Platform Mount
3	1	Ring Gear Lid
4	1	Upper Carrier
5	6	CCW Gear
6	1	Lower Carrier
7	1	Motor Shaft Gear
8	1	Bottom
9	1	Ring Gear
10	6	Rod
11	1	NEMA 17
12	1	DC Barrel Jack Mount

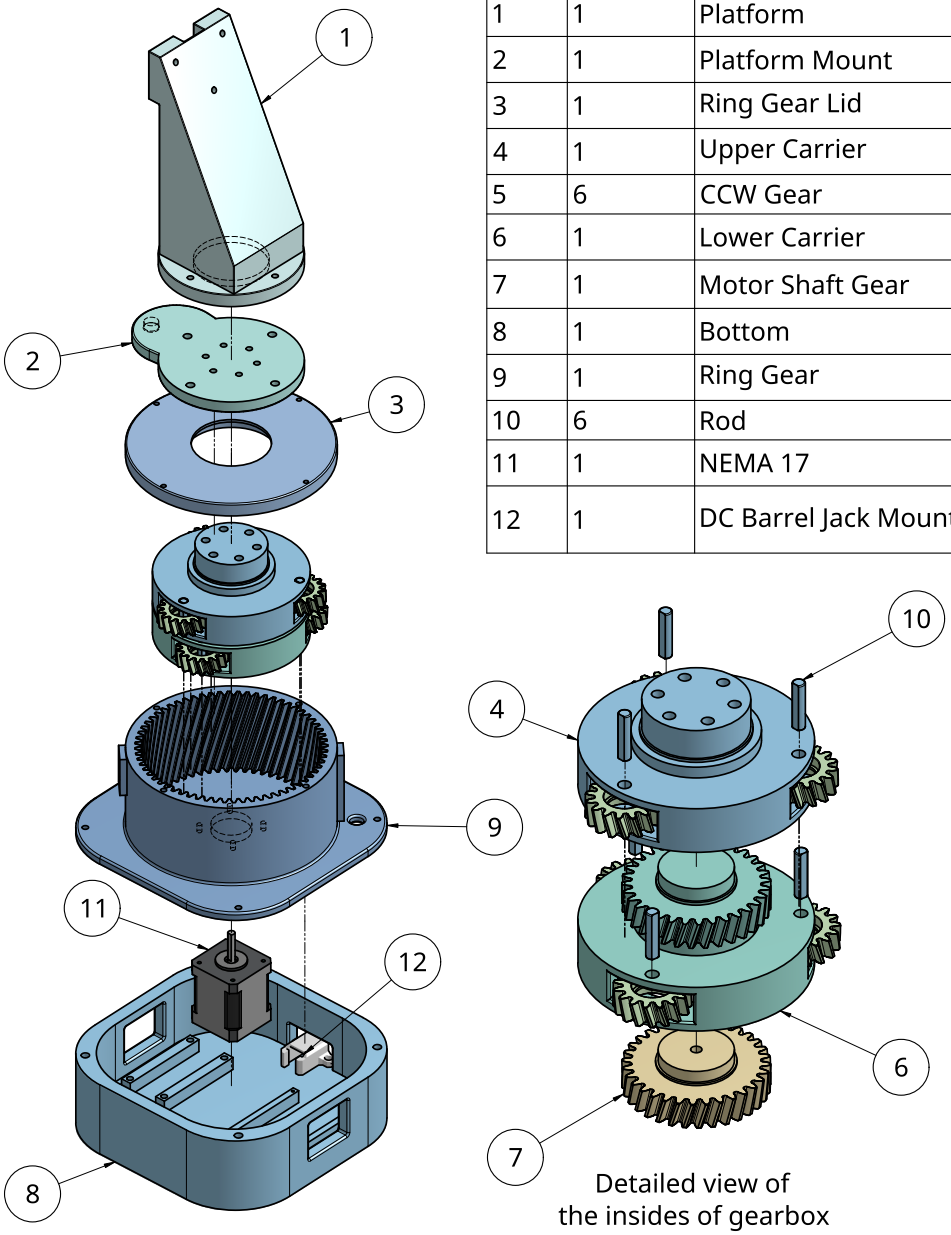


Figure 2.17: Exploded view of the platform

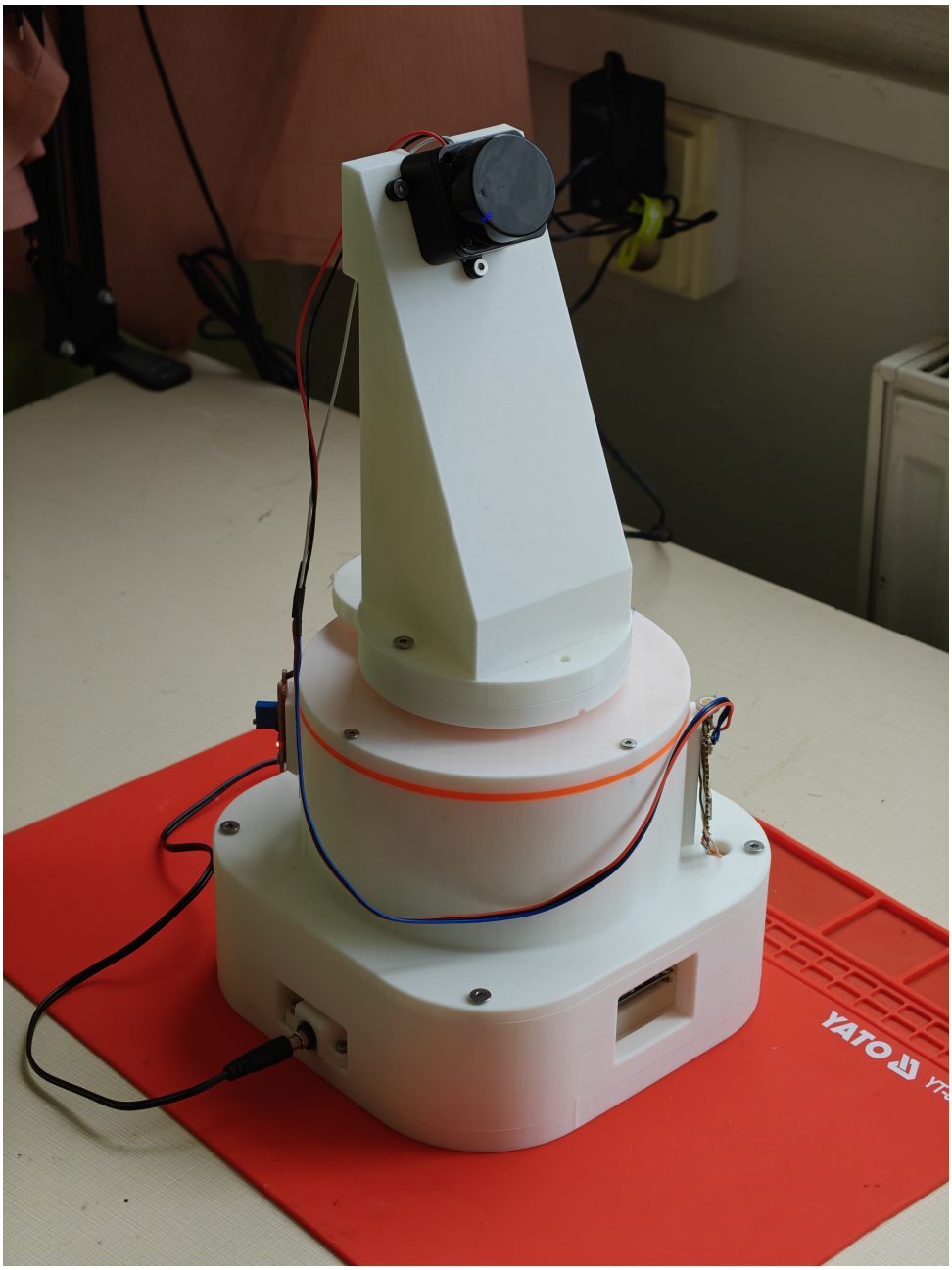


Figure 2.18: Photo of the assembled platform

Chapter 3

Electronics and Software Implementation

This chapter describes the electronic design and software implementation of data acquisition, parsing and processing.

3.1 Electronic Design

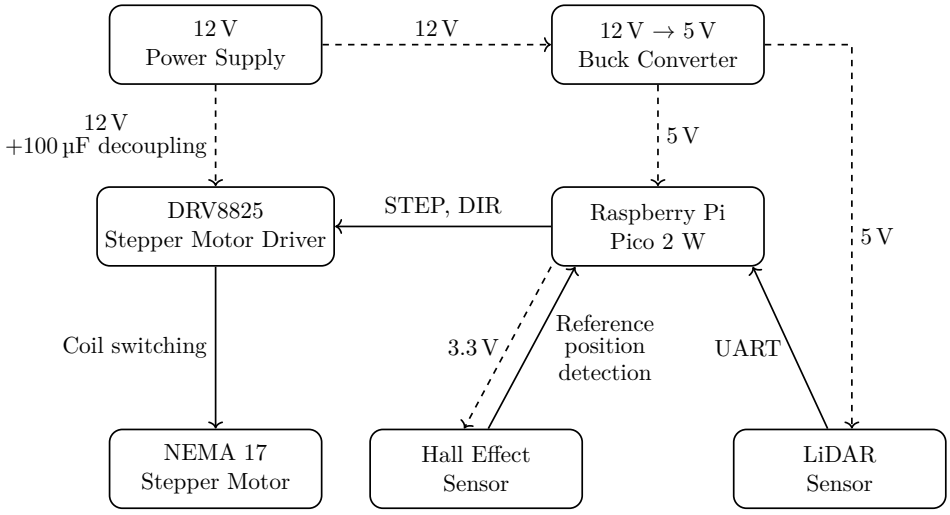


Figure 3.1: Block diagram of the electronic system. Dashed arrows represent power connections; solid arrows represent signal or control connections

The system is powered from an external 12 V power supply connected through a barrel jack connector. The 12 V powers the stepper motor driver and is simultaneously reduced to 5 V using a buck converter. The 5 V output is used to power the Raspberry Pi Pico 2 W and the LiDAR sensor.

The platform is rotated using a stepper motor mentioned in section 2.2.1 through the stepper motor driver. The driver receives control signals from the microcontroller via STEP and DIR pins. A 100 µF capacitor is connected across the driver's supply terminals in order to suppress voltage spikes generated during motor switching. RESET and SLEEP pins of the driver are tied together and connected to 5 V to keep the driver enabled at all times. A microstepping mode of 2:1 is set by connecting the MODE0 pin to 5 V and leaving MODE1 and MODE2 pins unconnected [18, sec. 8.3.5].

The LiDAR communicates with the microcontroller using a Universal Asynchronous Receiver-Transmitter (UART) RX pin. The acquired data are partially parsed by Pico and subsequently sent to a PC via Wi-Fi.

Additionally, a Hall effect sensor module is connected to the microcontroller and is used for the homing mechanism of a platform. This initialisation sequence is performed before each scan in order to prevent tangling of the wires connecting the LiDAR sensor to the microcontroller.

3.2 Software Implementation

The software part consists of two programs. The first one runs on Pico and is responsible for receiving commands from the PC, controlling the motor, acquiring data from the LiDAR and sending the data back to the PC. The second program runs on the PC and is responsible for determining the scanning density, receiving data from the Pico, parsing and processing the data, visualisation and storing of the point cloud.

3.2.1 Pico program

The Raspberry Pi Pico 2 W is programmed in Micropython. At the start of the program, the hardware is initialised and a Wi-Fi access point is created for the PC to connect to. After the connection is established, Pico waits for the PC to send the scanning density configuration, which is used to calculate the number of motor steps per scan.

Before the scanning starts, the platform is rotated until the reference position is detected by the Hall effect sensor. Then the main loop of the program starts. Each loop iteration is triggered by a command from the PC. When the Pico receives the `r` command, it starts acquiring data from the LiDAR for 200 ms[1] and after that parses each valid LiDAR packet, storing

[1]Even though the rotational frequency of the LiDAR sensor is approximately 10 Hz, corresponding to one full scan every 100 ms, an acquisition time of 200 ms was chosen in order to reliably obtain a complete scan slice even in the presence of possible timing delays.

only the necessary information needed for reconstructing the point cloud. After that it sends the data to the PC with a `done` signal to indicate the end of the scan, and moves the platform to the next position.

Pico program pseudocode

```
Initialise hardware
Initialise Wi-Fi access point
Wait for PC connection
Receive scan configuration from PC
Calculate number of motor steps per scan
Send ACK to PC
Perform platform homing
while program is running do
    Wait for command from PC
    if command is r then
        Mark start time
        while elapsed time is less than acquisition threshold do
            Read data from LiDAR UART
            Parse valid LiDAR packets
            Send compact packets to PC
        end while
        Send done to PC
        Move platform to next position
    else if command is stop then
        Stop motor
        Send restarting to PC
        Restart program
    end if
end while
```

3.2.2 PC program

The PC program is written in Python. Since our application is mostly IO-bound, it is divided into two threads.

The receiver thread is responsible for communication with the Pico. It sends the trigger command to the Pico, receives the data, parses it and performs the necessary transformations to obtain the point cloud. It also computes the current azimuth angle of the platform based on the number of requests already sent to the Pico (platform makes a step after every request) and the scanning density. The obtained points are then put into a global FIFO container.

The main thread is responsible for visualisation. It creates an Open3D visualisation window and constantly checks the global FIFO container for new

data. If there are new points, it updates the visualisation with the new data. Such separation allows the visualisation window to be responsive to user input even while the data is being received and processed in the background.

PC main thread pseudocode

```
Connect to Pico via Wi-Fi
Send the scanning density config (# of steps/scan) and wait for ACK
Start receiver thread
Initialise visualisation window
Initialise Open3D point cloud containers for visualisation and export
while acquisition is running do
    Read all available data from the global container into a local container
    if local container is not empty then
        Update visualisation with the new data
        if # of points in visualisation container > threshold then
            downsample visualisation container
        end if
    end if
    Update visualisation window with the current visualisation container
end while
Stop receiver thread
Send stop command to Pico and wait for restarting message as an ACK
Save export container on a machine
Destroy visualisation window
```

PC receiver thread pseudocode

```
for each platform position do
    Compute current azimuth angle
    Send trigger command to Pico r
    repeat
        Receive data chunk from Pico
        Append data chunk to a receive buffer
    until done signal received
    Parse data
    Perform transformation on the data (Polar -> Cartesian; 2D to 3D via
platform tilt angle and azimuth angle)
    Put obtained points into a global container
end for
```

3.2.3 Point cloud reconstruction

The data received from the Pico are a stream of trimmed LiDAR packets, containing only the start and stop angles and twelve distance measurements

"sandwiched" between them. The angle corresponding to each distance measurement is obtained by linear interpolation between the start and stop angles[2]. Then, the obtained polar coordinates are transformed into Cartesian coordinates in a 2D plane:

$$x = r \cdot \cos(\alpha), \quad y = r \cdot \sin(\alpha), \quad z = 0, \tag{3.1}$$

where r is the distance measurement and α is the corresponding linearly interpolated angle. Since our LiDAR performs only 2D scanning, all points initially lie in the xy-plane ($z = 0$).

The obtained points are then transformed according to the current orientation of the LiDAR. First, the points which are lying in xy-plane are rotated around x-axis by the angle ϕ, corresponding to the platform tilt angle (hard-coded constant in PC program):

$$\begin{bmatrix} x_\mathrm{t} \\ y_\mathrm{t} \\ z_\mathrm{t} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\phi & -\sin\phi \\ 0 & \sin\phi & \cos\phi \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix}. \tag{3.2}$$

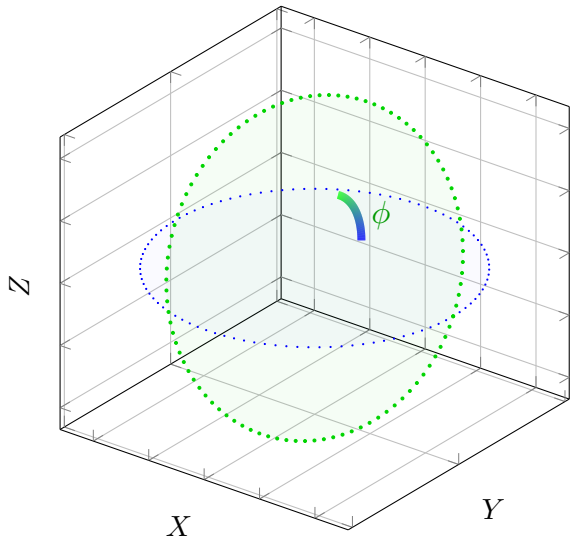


Figure 3.2: Visualisation of the tilt transformation

After the tilt transformation, the points are rotated around the z-axis by the angle θ, corresponding to the current platform azimuth:

$$\begin{bmatrix} x_\mathrm{a} \\ y_\mathrm{a} \\ z_\mathrm{a} \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_\mathrm{t} \\ y_\mathrm{t} \\ z_\mathrm{t} \end{bmatrix}. \tag{3.3}$$

[2] 360° is added to the stop angle in a case where the stop angle is less than the start angle due to the wrap-around occurring between 359° and 0°

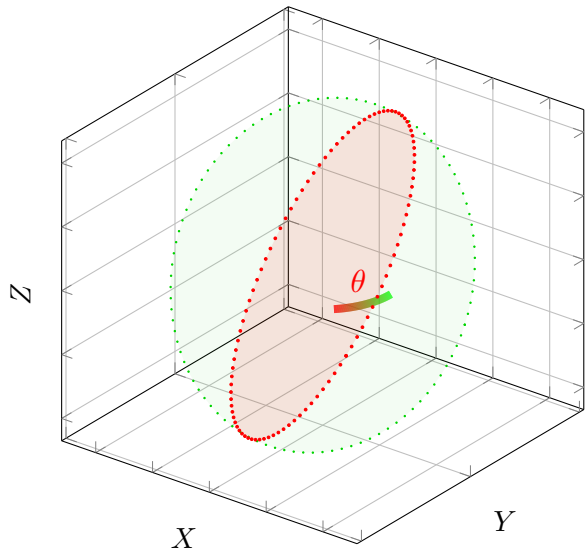


Figure 3.3: Visualisation of the azimuth transformation

By repeating the above process for each received packet from LiDAR, we can reconstruct the environment in the form of a point cloud.

Chapter 4

Experimental Results

4.1 Scanning procedure

In order to evaluate the performance of the platform, several scans of the author's room at the student dormitory were performed. The angular spacing between consecutive platform positions was set to 0.8°. With this setting, the total scanning time was approximately two minutes.

A video showing the scanning procedure is available at this link. A screen recording of the Open3D visualisation of the point cloud being built in real time is available at this link.

4.2 Point Cloud Processing

4.2.1 Outlier Removal

The obtained raw point cloud is shown in fig. 4.1. The overall boxy shape of the room is already visible; however, many spurious points can be seen far away from the scanned environment. These points are a result of faulty LiDAR measurements.

Since these faulty measurements are spatially separated from the main point cloud, they can be removed using outlier filtering tools provided by Open3D[1].

[1]Link: `https://www.open3d.org/docs/release/tutorial/geometry/pointcloud_outlier_removal.html`

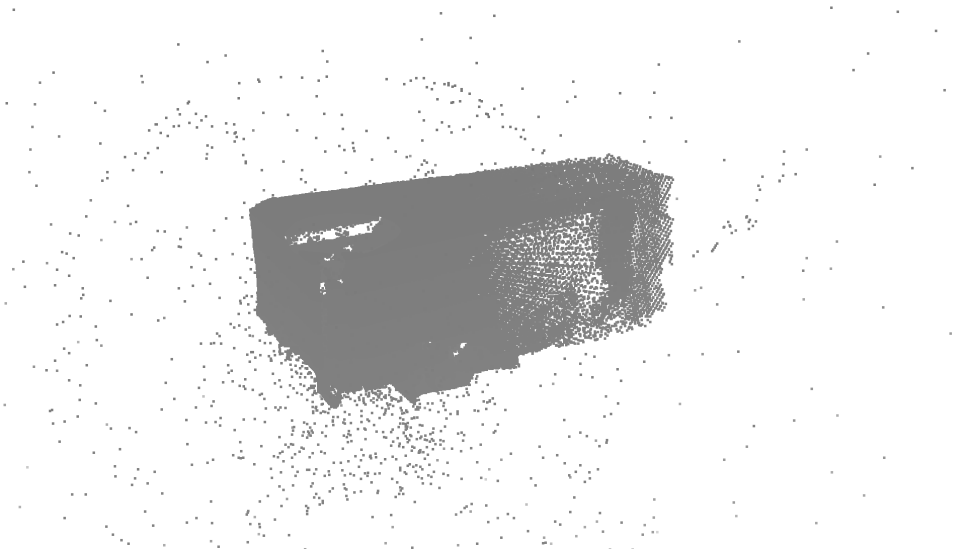
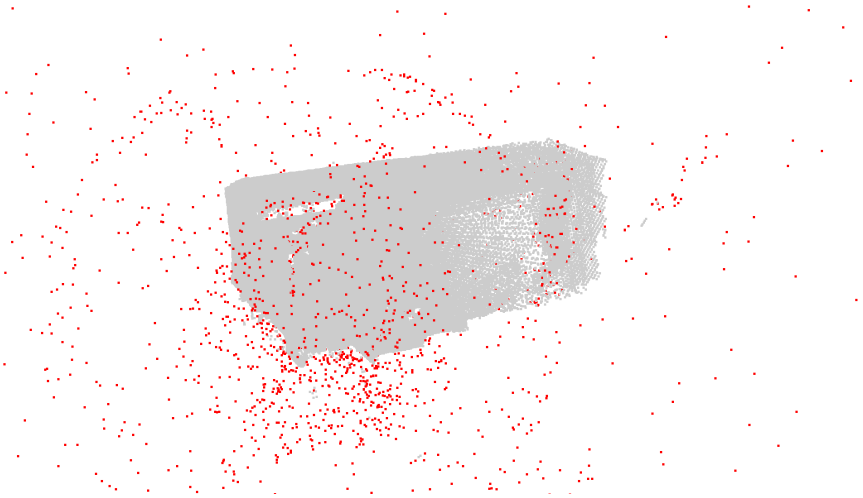
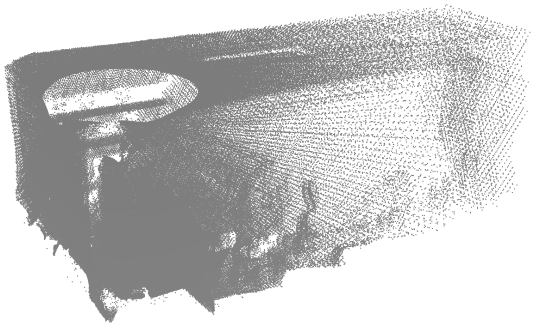


Figure 4.1: Raw point cloud



(a) : Red points are going to be removed



(b) : Filtered point cloud

Figure 4.2: Visualisation of the outlier removal process

4.2.2 Misalignment Corrections

After filtering, misalignments between individual scan slices became visible. These misalignments were most noticeable at wide planes, such as the ceiling and the walls and were made by the reconstruction from LiDAR scan slices acquired at platform azimuth angles differing by 180°. The exact cause of these misalignments is not known; however, two possible explanations are proposed, each of which was corrected separately.

The first correction was aimed at the misalignment of the ceiling. The mathematical transformation presented at section 3.2.3 assumes the x-axis of internal coordinate system of the LiDAR sensor to be perfectly horizontal. However, it is possible that the holes on the platform used for mounting the sensor were not perfectly level, causing a small angular error in the orientation of the sensor. To compensate for this effect, an experimental correction of $-1.0°$ was found to produce the best alignment of the ceiling when applied to every acquired scan slice before the tilt transformation.

Code snippet with the compensation for assumed LiDAR misalignment

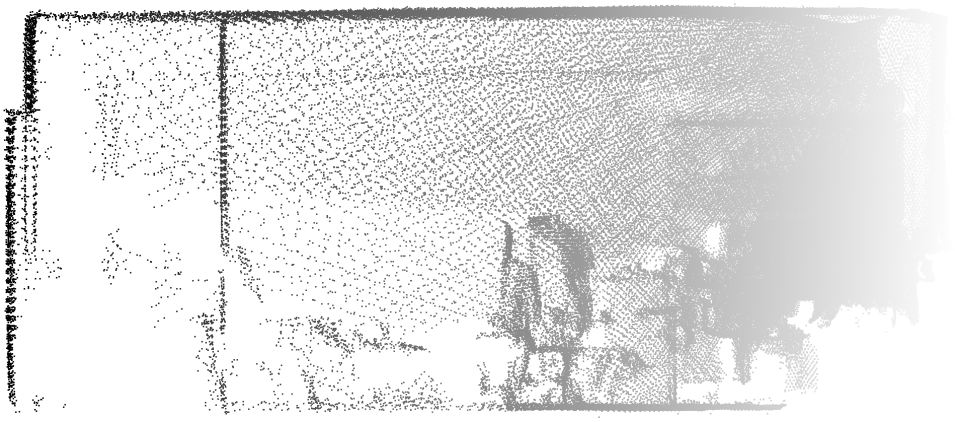
```
    def lidarTo3D(distance_mm, scan_angle_deg, azimuth_deg):
        r = distance_mm/1000. # mm -> m
        scan_angle_rad = np.deg2rad(scan_angle_deg-1.0) ###
         ↪  correction of the possible lidar misalignment ###
        tilt = np.deg2rad(PLATFORM_TILT_DEG)
        azimuth_rad = np.deg2rad(azimuth_deg)

        # lidar local 2d plane
        x = r*np.cos(scan_angle_rad)
        y = r*np.sin(scan_angle_rad)
        z = 0

        # tilt around x axis
        x_t = x
        y_t = y * np.cos(tilt) - z * np.sin(tilt)
        z_t = y * np.sin(tilt) + z * np.cos(tilt)

        # azimuth rotation around z
        x_a = x_t * np.cos(azimuth_rad) - y_t *
         ↪  np.sin(azimuth_rad)
        y_a = x_t * np.sin(azimuth_rad) + y_t *
         ↪  np.cos(azimuth_rad)
        z_a = z_t
        return [x_a,y_a,z_a]
```

(a) : Before correction



(b) : After correction

Figure 4.3: Correction of the ceiling misalignment shown in side view

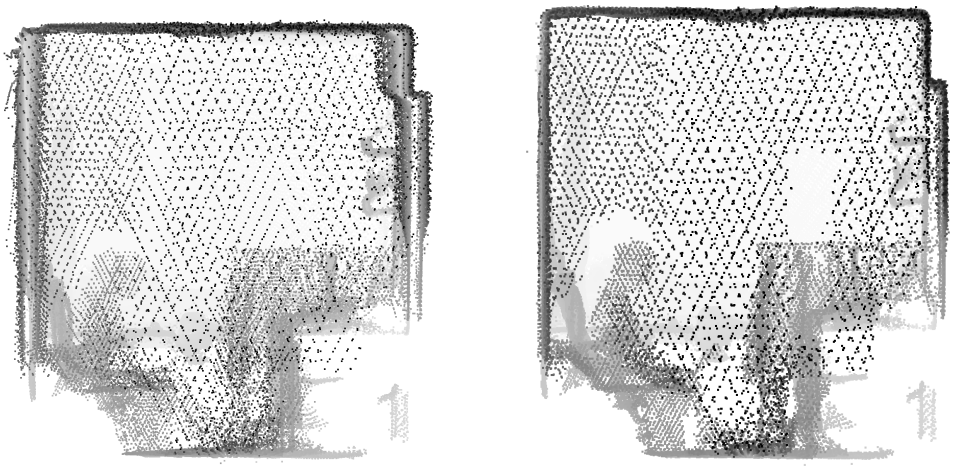
The second correction was aimed at the misalignment of walls. It was assumed that the LiDAR beam was not emitted exactly in the expected local plane, rather at a small vertical angle. This means that an acquired slice is not a slice but a collection of points shifted up vertically by an amount proportional to the measured distance. An experimental correction of 1.5° was found to produce the best alignment of the walls when applied before the tilt transformation.

Code snippet containing two corrections[2]

```python
def lidarTo3D(distance_mm, scan_angle_deg, azimuth_deg):
    r = distance_mm/1000. # mm -> m
    scan_angle_rad = np.deg2rad(scan_angle_deg-1.0)
    tilt = np.deg2rad(PLATFORM_TILT_DEG)
    azimuth_rad = np.deg2rad(azimuth_deg)

    # lidar local 2d plane
    x = r*np.cos(scan_angle_rad)
    y = r*np.sin(scan_angle_rad)
    z = r*np.sin(np.deg2rad(1.5)) ### correction of the possible lidar shooting angle ###

    ... # rest of the code is the same as in the previous snippet
```



(a) : Before correction **(b) :** After correction

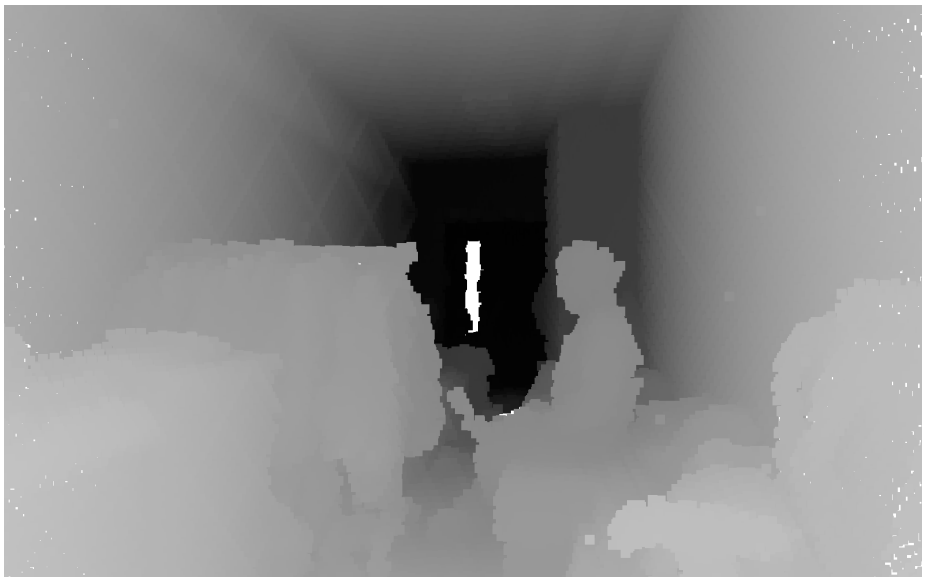
Figure 4.4: Correction of the wall misalignment shown in side view

4.2.3 Comparison with Photograph

For this demonstration, the size of surfels in the Open3D visualisation window was set to maximally large in order to make the point cloud more visible and give a feeling of the continuity of surfaces.

[2]To be mathematically correct, both x and y should also be multiplied by `np.cos(np.deg2rad(1.5))`. However, for such a small angle, the cosine is close to one and the effect of this correction on x and y would therefore have been negligible.

(a) : Photo



(b) : Point cloud

Figure 4.5: Comparison between a photo and the point cloud of a scanned room

The grayscale colouring is based on one selected coordinate component of the points in the Open3D visualisation, which helps convey the spatial depth of the reconstructed scene. Author is visible sitting on a bed. Points which were reflected by mirror were positioned at a greater distance due to the reflection of the LiDAR beam and were therefore filtered out by the outlier removal process.

Chapter 5

Conclusion

The aim of this thesis was to design a cost-effective solution for 3D spatial scanning using a more affordable 2D LiDAR sensor. Existing amateur solutions were first analysed and compared, and a rotating tilted platform design was proposed that addresses the most significant limitations of the existing approaches.

A planetary gearbox was introduced into the design in order to reduce the output angular step of the platform driven by a stepper motor, which allowed for a larger tilt angle and therefore better coverage of the scanned environment while still preserving sufficient point cloud density.

Data acquisition and platform control were implemented using a Raspberry Pi Pico 2 W, which drives the stepper motor through a motor driver, receives data from the LiDAR sensor and forwards the necessary measurements to a PC via Wi-Fi. The PC-side program reconstructs the point cloud from the received data and visualises the result using the Open3D library.

The functionality of the prototype was verified by scanning the room. The initial obtained point cloud contained several imperfections, namely: a large number of spurious points and misalignments between scan slices obtained at platform azimuthal angles differing by 180°. The spurious points were spatially separated from the useful part of the scan and were therefore easily removed using outlier filtering tools provided by Open3D. The misalignments were most noticeable on large planar surfaces such as the ceiling and walls and were compensated experimentally by tuning the transformation parameters in the PC program. After applying the corrections, the resulting point cloud showed good agreement with a photo of the scanned room.

Below is the total bill of materials for the constructed platform:

Table 5.1: Bill of materials

Part	Quantity	Price [€]
2D LiDAR sensor	1	80
Raspberry Pi Pico 2 W	1	7
Buck converter	1	4
12 V power supply	1	8
NEMA 17 17HS8401 stepper motor	1	14
DRV8825 stepper motor driver	1	4
Hall effect sensor	1	1
Magnet	1	1
100 µF capacitor	1	–
Ball bearing 15 × 24 × 5 mm	12	4
Ball bearing 50 × 65 × 7 mm	1	9
Ball bearing 35 × 47 × 7 mm	2	8
DC barrel jack adapter – 2-pin terminal block	1	–
PLA filament (700 g)	1	23
Perfboard	1	1
Other (wires, screws, threaded inserts)	–	–
Total		**164**

So, the total cost of the constructed platform is lower than the price of the cheapest commercial 3D LiDAR sensor mentioned in chapter 1, confirming the cost-oriented motivation of the thesis.

Although the prototype fulfilled its main purpose, several drawbacks of the current design were identified. The overall mechanical construction could have been made more compact if a smaller gear size was chosen. In our case, the size of the planet gears was determined by the size of available ball bearings and due to concerns about possible printing issues in the case of smaller gears. Screw holes meant to keep Pico and the hall effect sensor module at the outer wall of the ring gear came out too loose and so those components can be easily detached from the platform. The hole for the magnet also turned out to be too large, so the duct tape was used to prevent the magnet from falling out of the hole. Also, during the initial planning of the design, only a gearbox reduction ratio of 2:1 was considered. Therefore, an unnecessarily strong stepper motor was selected, which is larger and more expensive than its weaker alternatives. These limitations could be easily addressed in a future version of the platform.

Overall, the resulting prototype demonstrates that an affordable 2D LiDAR sensor, combined with a simple 3D-printed platform and suitable software processing, can be used to obtain a detailed three-dimensional point cloud, confirming the main objective of this thesis.

Appendix A

Bibliography

[1] DOUBRAVA, Pavel. *LIDAR senzor a jeho aplikace v mobilním robotu* [online]. 2021. [visited on 2025-11-23]. Available from HDL: 10467/94785. MA thesis. České vysoké učení technické v Praze.

[2] DONG, Pinliang; CHEN, Qi. *LiDAR Remote Sensing and Applications*. Boca Raton: CRC Press, 2017. ISBN 978-1-351-23335-4. Available from DOI: 10.4324/9781351233354.

[3] *4D Lidar L1 Application Scenarios_4D Lidar L1 Efficacy | Unitree Robotics* [online]. [visited on 2025-11-23]. Available from: https://www.unitree.com/LiDAR.

[4] ZHOU, Qian-Yi; PARK, Jaesik; KOLTUN, Vladlen. *Open3D: A Modern Library for 3D Data Processing* [online]. 2018-01-30. [visited on 2025-10-02]. Available from DOI: 10.48550/arXiv.1801.09847.

[5] PETERS, Dana. *LIDAR Scanner* [online]. 2022-01-19. [visited on 2025-11-17]. Available from: https://www.qcontinuum.org/lidar-scanner.

[6] GRASSIN, Charles. *3D Lidar Scanner MK2* [online]. Charles' Labs, 2020-10-14. [visited on 2025-10-02]. Available from: https://charleslabs.fr/en/project-3D+Lidar+Scanner+MK2.

[7] LINGKANG, Zhang. *DIY 3D Lidar* [online]. Lingkang Zhang, 2022-01-03. [visited on 2025-11-23]. Available from: http://zlethic.com/diy-3dlidar/.

[8] FRIEDL, František. *Speed control stepper*. 2008. Available from HDL: 10563/6878. BA thesis.

[9] BY. *How Accurate Is Microstepping Really?* [online]. Hackaday, 2016-08-29. [visited on 2026-05-07]. Available from: https://hackaday.com/2016/08/29/how-accurate-is-microstepping-really/.

[10] DEJAN. *What Is THE BEST 3D Printed Drive for Your Next Robotic Project* [online]. How To Mechatronics, 2025-03-31. [visited on 2026-05-14]. Available from: https://howtomechatronics.com/how-it-works/what-is-the-best-3d-printed-drive-for-your-next-robotic-project/.

[11] QI, Le; YANG, Dapeng; CAO, Baoshi; LI, Zhiqi; LIU, Hong. Design Principle and Numerical Analysis for Cycloidal Drive Considering Clearance, Deformation, and Friction. *Alexandria Engineering Journal* [online]. 2024, vol. 91, pp. 403–418 [visited on 2026-05-14]. ISSN 1110-0168. Available from DOI: 10.1016/j.aej.2024.01.077.

[12] DEJAN. *What Is Cycloidal Driver? Designing, 3D Printing and Testing* [online]. How To Mechatronics, 2021-12-19. [visited on 2026-05-14]. Available from: https://howtomechatronics.com/how-it-works/what-is-cycloidal-driver-designing-3d-printing-and-testing/.

[13] YOUNIS, Omar. Building a Cycloidal Drive with SOLIDWORKS [online]. [N.d.] [visited on 2026-05-15]. Available from: https://blog-assets.solidworks.com/uploads/2025/12/building-a-cycloidal-drive-with-solidworks.pdf.

[14] NAUNHEIMER, Harald; BERTSCHE, Bernd; RYBORZ, Joachim; NOVAK, Wolfgang. *Automotive Transmissions: Fundamentals, Selection, Design and Application* [online]. Berlin, Heidelberg: Springer, 2011 [visited on 2026-05-14]. ISBN 978-3-642-16213-8 978-3-642-16214-5. Available from DOI: 10.1007/978-3-642-16214-5.

[15] DEJAN. *How Planetary Gears Work - 3D Printed Planetary Gearbox Design and Test* [online]. How To Mechatronics, 2023-07-13. [visited on 2026-02-09]. Available from: https://howtomechatronics.com/how-it-works/how-planetary-gears-work-3d-printed-planetary-gearbox-design-and-test/.

[16] *Gear Systems | KHK Gear Manufacturer* [online]. [visited on 2026-05-15]. Available from: https://khkgears.net/new/gear_knowledge/gear_technical_reference/gear_systems.html.

[17] TEC-SCIENCE. *Transmission Ratios of Planetary Gears (Willis Equation) | Tec-Science* [online]. 2021-03-10. [visited on 2026-05-17]. Available from: https://www.tec-science.com/mechanical-power-transmission/planetary-gear/transmission-ratios-of-planetary-gears-willis-equation/.

[18] ALLDATASHEET.COM. *DRV8825 Download* [online]. [visited on 2026-05-15]. Available from: http://www.alldatasheet.com/datasheet-pdf/download/835822/TI1/DRV8825.html.

Appendix B

Attached files

The submitted attachments are:

- code.zip
- 3d_models.zip

B.1 code.zip

The structure of `code.zip` is as follows:

```
code
|-- README.md
|-- electronic_design.pdf
|-- examples
|   |-- pcd_2026.05.16_13:41:33_495468-badalign-nocompensation.ply
|   |-- pcd_2026.05.16_13:54:25_494384-badalign-minus1scanangle-
ceilingcorrect.ply
|   `-- pcd_2026.05.16_14:02:09_494400-best-minus1scanangle-plus1.5z.ply
|-- pc.py
|-- pcd_visualizer.py
|-- pcd_visualizer_outlier_removal.py
|-- pico_load
|   |-- main.py
|   |-- secrets.py
|   `-- stepper.py
`-- requirements.txt
```

- `README.md` contains instructions for setting up the Python `.venv` envi-

ronment and running the project.

- `electronic_design.pdf` is a wiring diagram created in Fritzing that shows the pin connections used in this project.
- `examples/` folder contains point clouds that were discussed in section 4.2.2.
- `pc.py` is the PC code responsible for communication with Pico, acquisition and parsing of LiDAR data, point cloud reconstruction and management of the visualisation window showing the real-time reconstruction.
- `pcd_visualizer.py` and `pcd_visualizer_outlier_removal.py` are scripts used for visualising the stored point clouds
- `pico_load/` folder contains code intended to be uploaded to Pico. `secrets.py` should be filled with the preferred Wi-Fi SSID and password used for connecting to the Pico access point.
- `requirements.txt` contains Python libraries needed for running the code on PC.

B.2 3d_models.zip

```
3d_models
|-- Bottom.stl
|-- CCW Gear x6.stl
|-- DC housing.stl
|-- Lower Carrier.stl
|-- Motor Shaft Gear.stl
|-- Platform Mount.stl
|-- Platform.stl
|-- Ring Gear Lid.stl
|-- Ring Gear.stl
|-- Rod x6.stl
|-- Upper Carrier.stl
`-- model_attribution.txt
```

The `3d_models` contains all 3D models in `.stl` format used for building the platform. The suffix `x6` in some filenames indicates the corresponding number of copies required for the assembly.

The `DC housing.stl` is not author's work. The attribution link is provided in `model_attribution.txt`.

The 3D modelling was done in Onshape. The complete assembly can be accessed via this link.

B.3 Project Repository

The entire development of this thesis was managed using Git. The repository is hosted on the faculty GitLab server and can be accessed via this link.